Implementing Conditional Access on FPGA

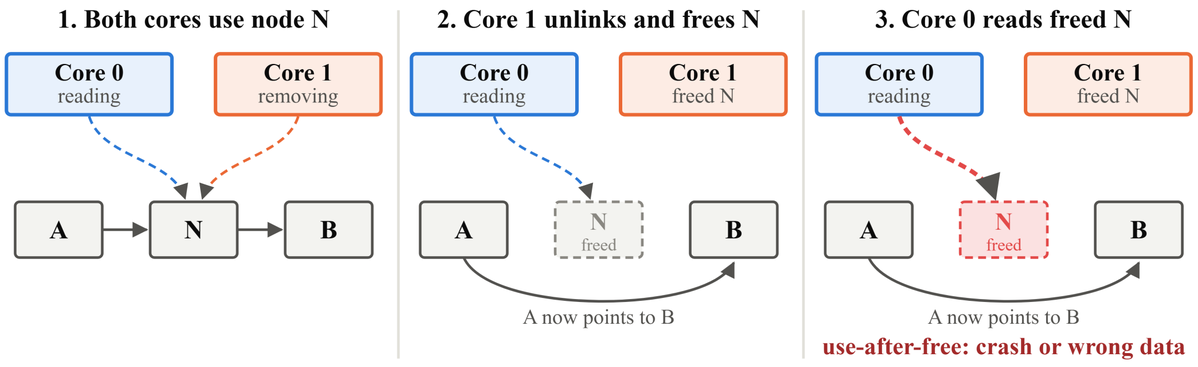
A dual-core, cache-coherent RISC-V processor that frees memory immediately

Aryan Kumar, Nishant Dhankhar, Kudumula Vinod Kumar Reddy *Advisor:* Ajay Singh

Department of Computer Science and Engineering, IIT Gandhinagar

1 The problem: use-after-free

Lock-free data structures let threads read nodes that other threads are removing.



2 Today's fixes delay freeing

Hazard pointers

announce p → fence → read *p

on every pointer read

Problem: a memory fence on every read slows traversals

Epoch-based (EBR)

removed, waiting

freed only after every thread passes a checkpoint

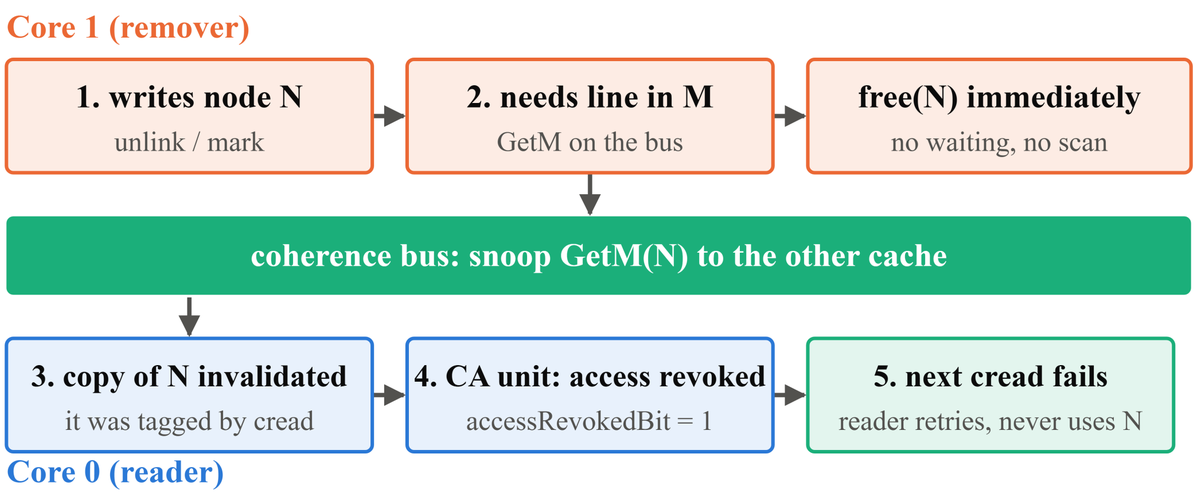
Problem: unfreed memory piles up, without bound if a thread stalls

Both delay reclamation: they trade speed against memory

3 Conditional Access (CA)

New instructions [1] that reuse the cache-coherence protocol: a write by another core **revokes** the reader's access, so memory can be freed **immediately**.

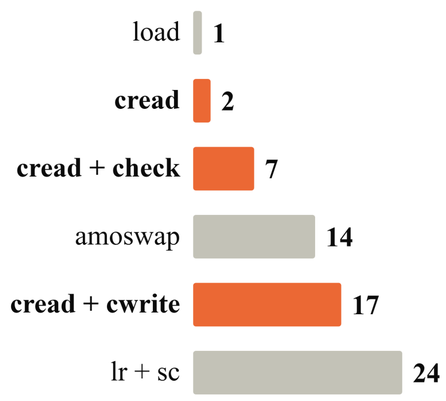
`cread`	`cwrite`	`untagOne`	`untagAll`
load and watch the line	store if nothing watched changed	stop watching one line	stop watching all, clear flags



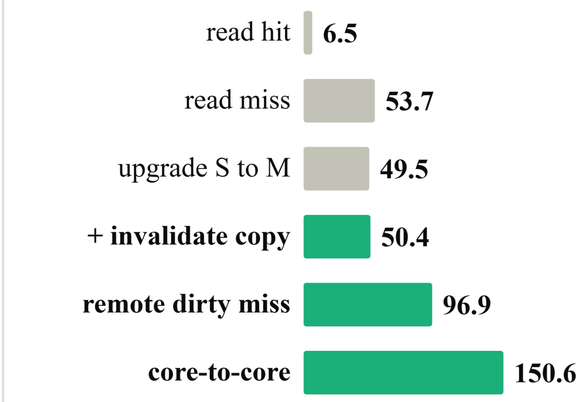
4 Why build it on an FPGA?

CA was only simulated. Real hardware answers what a simulator cannot: the **latency** of the instructions, the **cost of the coherence events** CA relies on, and the **area** it needs. Measured on the KR260:

Instruction latency (cycles)

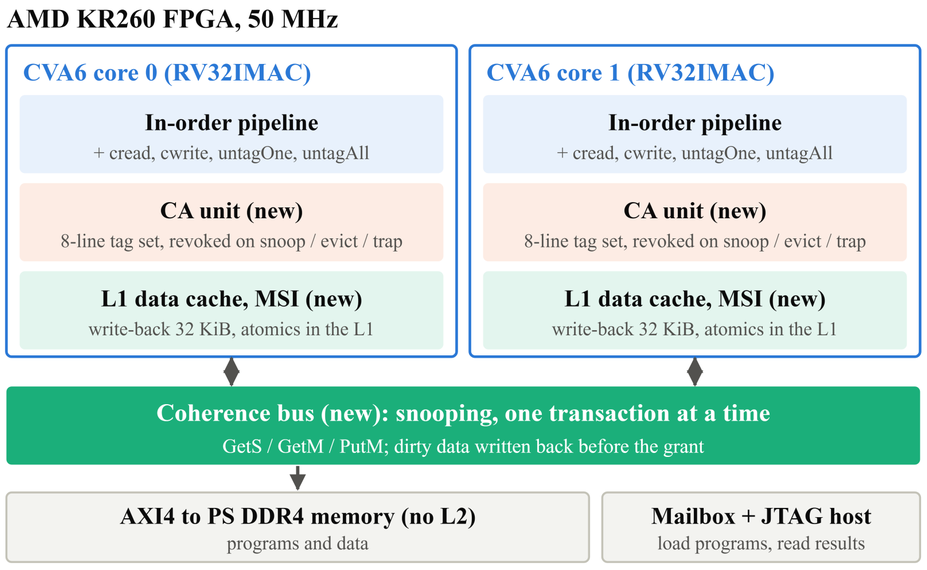


Coherence cost (cycles per line)



invalidating the other copy adds only ~1 cycle

5 What we built on CVA6 [2]



6 Challenges

Choosing a base
OpenPiton dropped: old CVA6, heavy, needs an L1.5 adapter

Fitting the FPGA
limited LUTs: moved to 32-bit RV32 cores

Deadlock freedom
a cache answers a snoop without needing the bus

Atomic cwrite
runs at commit, holding the line exclusively

Speculation
CA ops recorded at issue, applied only at commit

Learning curve
CVA6, KR260 and Vivado, coherence, reclamation

7 Verification

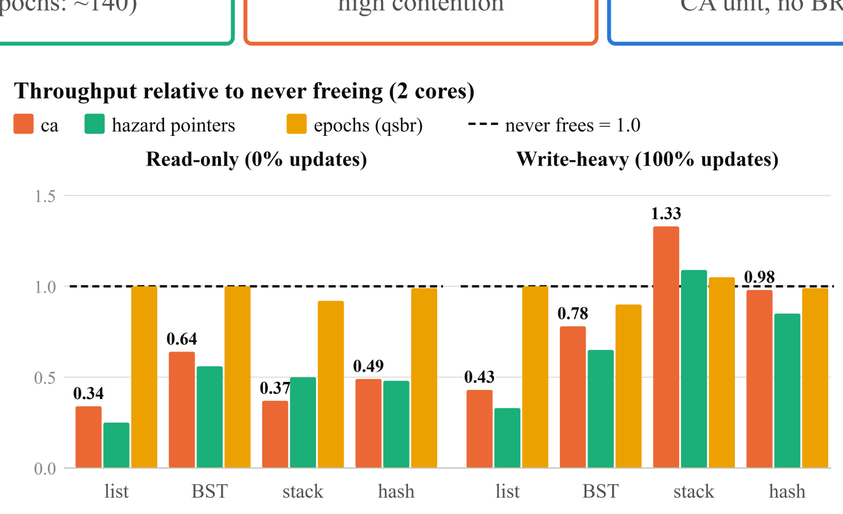
One cache — random traffic vs reference memory	300 / 300
Two caches + bus + CA — models of MSI and of the CA rules	400 / 400
Injected bugs — does the testbench catch them?	26 / 27
Full processor — RISC-V ISA test suites	72 / 73
KR260 board — two-core programs, same as simulation	20 / 20

8 Results on the board

0
nodes left unfreed (epochs: ~140)

1.33×
throughput under high contention

4.5%
of a core for the CA unit, no BRAM



9 Conclusion

CA works on real hardware: immediate freeing, fastest under contention, 4.5% of a core. **Next:** a branch-on-failure instruction.

[1] A. Singh, T. Brown, M. Spear. Efficient hardware primitives for immediate memory reclamation in optimistic data structures. arXiv:2302.12958, 2023. [2] F. Zaruba, L. Benini. The cost of application-class processing. *IEEE TVLSI*, 2019.

Acknowledgements. We thank Prof. Ajay Singh for his guidance throughout this project, and the Department of Computer Science and Engineering, IIT Gandhinagar, for access to the AMD KR260 board and tools.